



取回线程的工作结果

北京理工大学计算机学院
金旭亮

取回线程的工作结果

Runnable接口所定义的run()方法没有返回值，因此，不能直接将结果返回给调用者。

JDK 1.5引入了一个Callable接口，它可以返回值给调用者，并且在无法得到结果时会抛出一个异常：

```
@FunctionalInterface
public interface Callable<V> {
    /**
     * Computes a result, or throws an exception if unable to do so.
     *
     * @return computed result
     * @throws Exception if unable to compute a result
     */
    225 implementations
    V call() throws Exception;
}
```

注意一下这个接口是一个函数式接口，因此，它定义的变量可以接收一个Lambda表达式。

使用Callable封装后台计算任务

```
//定义一个将被工作线程执行的，有返回值的任务
Callable<String> task = () -> {
    var begin :long = System.currentTimeMillis();
    for (int i = 0; i < 10; i++) {
        Thread.sleep( millis: 500);
        System.out.print(".");
    }
    var elapsed = System.currentTimeMillis() - begin;
    return "工作结束,耗时" + elapsed + "毫秒";
};
```

GetThreadResult.java

启动线程进行后台计算

```
//Thread不能直接接收一个Callable对象，因此，需要再包一层  
var job = new FutureTask<>(task);  
//创建线程并启动执行  
new Thread(job).start();  
//FutureTask所定义的get()方法用于取回结果，它会在此“阻塞等待”.....  
System.out.println("\nresult:" + job.get());
```



```
Run: GetThreadResult x  
"C:\Program Files\Java\jdk-17\bin\java.exe"  
.....  
result:工作结束,耗时5086毫秒  
Process finished with exit code 0
```

巩固练习



使用`GetThreadResult`示例所展示的`Callable`编程模式，编写一个示例，使用多线程计算1到100的和，显示最终结果。

取回线程工作结果的另一种方式—回调



前面示例所展示的编程模式，是主线程主动地阻塞等待工作线程完成计算任务，这种方式，会导致主线程的阻塞，可能影响到多线程程序的性能。



换用另外一种思路，主线程“事先”准备好一个函数，“告诉”工作线程：“你努力干活，活干完之后，调用我告诉你的那个函数，再把你的工作结果作为参数传给那个函数”。
完成这些工作之后，主线程就“自顾自”地去工作，不再理会这件事情了。



这种编程模式，就是“回调（call back）”，“那个函数”，即负责处理最终结果的函数，被称为“回调函数”。

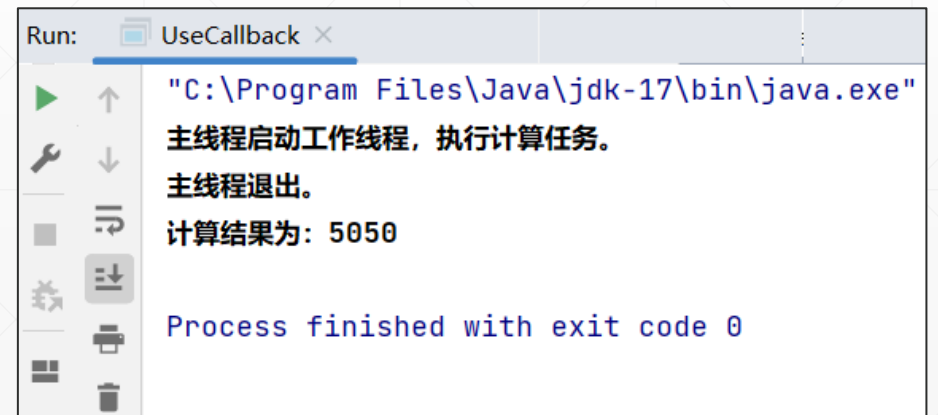
回调示例

```
UseCallback.java x
1  ▶ public class UseCallback {
2
3  ▶     public static void main(String[] args)
4      throws InterruptedException {...}
19
20     //供工作线程回调的函数，此函数负责处理工作线程的工作结果
    1 usage
21     public static void callback(long result) {
22         System.out.println("计算结果为: " + result);
23     }
24 }
```

回调的实现

```
public static void main(String[] args)
    throws InterruptedException {
    //一个简单的计算任务, 求1到100的和
    Runnable task = () -> {
        long sum = 0;
        for (int i = 1; i <= 100; i++) {
            sum += i;
        }
        //结果出来之后, 调用回调函数显示最终结果
        callback(sum);
    };
    System.out.println("主线程启动工作线程, 执行计算任务。");
    var worker = new Thread(task);
    worker.start();
    System.out.println("主线程退出。");
}
```

回调这种编程技巧, 极为重要, 请注意掌握。



```
Run: UseCallback x
"C:\Program Files\Java\jdk-17\bin\java.exe"
主线程启动工作线程, 执行计算任务。
主线程退出。
计算结果为: 5050
Process finished with exit code 0
```

巩固练习



编写一个方法，这个方法可以等待指定长的时间，然后，执行一个用户指定的工作任务。

提示：

可以写一个方法，这个方法接收两个参数，第一个参数代表要等待的时间，第二个参数是一个函数式接口，它用于引用外部动态传入的Lambda表达式，这个Lambda表达式代表用户要执行的工作任务。

在方法内部启动一个线程，在线程函数中使用`Thread.sleep()`方法暂停指定的时间，时间到后，执行外部传入的Lambda表达式